

Tối ưu hóa Retrieval-Augmented Generation cho dữ liệu có cấu trúc cao: Chiến lược Chunking nâng cao và ngữ cảnh hóa trong xử lý hồ sơ (Resume/CV)

Tính tất yếu về kiến trúc trong Retrieval-Augmented Generation

Các kiến trúc Retrieval-Augmented Generation (RAG) đã thay đổi căn bản năng lực vận hành của các Mô hình Ngôn ngữ Lớn (LLM) bằng cách “neo” đầu ra sinh văn bản vào các kho tri thức bên ngoài, chuyên biệt theo miền.¹ Chu trình truy hồi tiêu chuẩn thường diễn ra tuần tự: nạp tài liệu (ingestion), tạo vector embedding, lập chỉ mục cơ sở dữ liệu, truy hồi ngữ nghĩa, tăng cường prompt, và cuối cùng là sinh câu trả lời.² Trong kiến trúc này, giai đoạn ingestion và chunking là yếu tố quyết định nhất đối với độ chính xác ở các bước phía sau.⁴ Lý thuyết vận hành dựa trên giả định rằng việc chia các kho văn bản lớn thành các vector nhỏ hơn nhưng giàu ngữ nghĩa cho phép mô hình embedding ánh xạ truy vấn của người dùng tới các điểm dữ liệu “gần” nhất về mặt toán học trong không gian tiềm ẩn nhiều chiều.⁶

Tuy nhiên, hiệu quả của mô hình này bị thử thách nghiêm trọng khi áp dụng cho các tài liệu có cấu trúc cao và mật độ thông tin lớn, chẳng hạn như Resume/CV được định dạng JSON.⁷ Trong khi các hệ thống truy hồi truyền thống xử lý tốt văn bản tự sự liên tục—như bài báo, hợp đồng pháp lý, hay quy trình vận hành chuẩn—chúng lại thường xuyên gặp các “chế độ lỗi” mang tính thảm họa khi phải phân tích các cấu trúc phân cấp.⁸ Resume là một dạng dữ liệu đặc thù với các khẳng định thực tế dày đặc, phụ thuộc theo dòng thời gian, và các quan hệ thực thể ngầm.¹⁰ Khi các tài liệu này được tuần tự hóa thành các mảng/đối tượng JSON, ý nghĩa ngữ nghĩa của từng mẫu dữ liệu gắn chặt với vị trí của nó trong hệ phân cấp tài liệu. Một thành tích chi tiết như “giảm độ trễ máy chủ 40%” sẽ gần như vô dụng nếu bị tách rời khỏi các nút cha trong JSON—những nút xác định danh tính ứng viên, nhà tuyển dụng cụ thể, và thời gian đảm nhiệm vai trò.⁵

Khi các hệ thống doanh nghiệp mở rộng để xử lý hàng triệu hồ sơ ứng viên phục vụ tuyển dụng, mâu thuẫn giữa việc chunking thật nhỏ và việc giữ ngữ cảnh toàn cục trở thành thách thức kỹ thuật trọng tâm.¹¹ Nếu không giải quyết được mâu thuẫn này, hệ thống sẽ truy hồi các “mảnh mò côi” (orphan chunks)—những đoạn bị cô lập về ngữ nghĩa—dẫn tới ảo giác nghiêm trọng hoặc khiến mô hình sinh trả lời thiếu sót, sai lệch.¹² Báo cáo này đưa ra phân tích so sánh toàn diện về các phương pháp phân đoạn tài liệu cho dữ liệu JSON có cấu trúc. Nó đánh giá cơ chế và hiệu năng của các bộ chia văn bản ngây thơ (naive text splitters) so với chunking theo nút cấu trúc (node-based structural chunking) được tăng cường bằng kỹ thuật bơm ngữ cảnh (context enrichment) theo chương trình. Ngoài ra, báo cáo mổ xẻ các khuôn mẫu tiên tiến—đặc biệt là Contextual Retrieval của Anthropic và Late Chunking của Jina AI—và tổng hợp một khung thực hành tốt nhất

trong ngành nhằm bảo toàn ranh giới ngữ nghĩa khi vector hóa mà không phát sinh chi phí suy luận (inference) quá lớn ở giai đoạn chuẩn bị dữ liệu.

Độ phức tạp toán học và cấu trúc của JSON trong không gian vector

Để đánh giá có hệ thống các chiến lược chunking, cần phân tích tương tác cơ học giữa dữ liệu JSON có cấu trúc cao và kiến trúc tokenization của các mô hình embedding hiện đại. Các thuật toán embedding—thường xây dựng trên BERT hoặc các kiến trúc transformer hai chiều tương tự—được huấn luyện trên kho dữ liệu khổng lồ gồm văn bản tự nhiên liên tục nhằm nắm bắt ý nghĩa ngữ nghĩa sâu.¹⁴ Chúng sử dụng các thuật toán tách token như Byte-Pair Encoding (BPE) hoặc WordPiece, vốn được tối ưu mạnh cho văn xuôi của con người.¹⁴

Khi dữ liệu JSON thô (chưa được phân tích/chuẩn hóa) được đưa vào các bộ tokenizer này, hệ thống sẽ gặp mặt độ rất lớn các ký tự phi chữ-số như ngoặc nhọn, ngoặc vuông, dấu hai chấm và dấu ngoặc kép. Trong một resume dạng JSON, một cặp khóa-giá trị đơn giản như "years_of_experience": 8 không được xử lý như một đơn vị ngữ nghĩa gắn kết. Thay vào đó, tokenizer băm vụn chuỗi thành nhiều token rời rạc: tách riêng dấu ngoặc kép, dấu gạch dưới, dấu hai chấm và giá trị số.¹⁴ Sự phân mảnh này làm suy giảm mạnh tỉ lệ tín hiệu/nhiều trong cửa sổ ngữ cảnh của mô hình embedding. Trong văn bản tiếng Anh chuẩn, gần như mọi token đều đóng góp vào tín hiệu ngữ nghĩa; còn trong JSON thô, một phần đáng kể giới hạn token bị “phung phí” cho cú pháp cấu trúc hầu như không mang giá trị ngữ nghĩa cho biểu diễn vector.¹⁴

Không chỉ kém hiệu quả về tokenization, chức năng lõi của transformer còn dựa vào cơ chế self-attention để cân trọng số tầm quan trọng tương đối của token trong toàn chuỗi.¹⁴ Trong JSON lồng nhau sâu, “khoảng cách ngữ nghĩa” giữa một nút con ở sâu và nút gốc có thể kéo dài hàng nghìn token cú pháp.¹⁶ Ví dụ, tên ứng viên có thể nằm ở token thứ 10, trong khi một thành tích kỹ thuật cụ thể nằm ở token thứ 3.500. Nếu một thuật toán chunking tùy tiện cắt tài liệu giữa hai vị trí này, self-attention sẽ bị “mù” trước quan hệ giữa chúng.¹⁷ Vector embedding thu được chỉ còn là một cụm từ mất ngữ cảnh, “trôi” trong không gian tiềm ẩn mà không neo vào danh tính ứng viên—khiến nó gần như vô hình đối với các truy vấn rất cụ thể.¹⁸

Bảng 1. So sánh xử lý ngôn ngữ tự nhiên và xử lý JSON thô

Yếu tố	Xử lý ngôn ngữ tự nhiên	Xử lý JSON thô
Hiệu quả tokenization	Cao; thuật toán nhóm các thành phần con phổ biến của từ. ¹⁴	Thấp; cú pháp bị băm vụn thành các token vô nghĩa. ¹⁴
Tỉ lệ tín hiệu / nhiễu	Cao; đa số token mang	Thấp; chi phí cú pháp cao

	trọng lượng ngữ nghĩa.14	làm loãng ý nghĩa.14
Cơ chế attention	Dễ liên kết danh từ và động từ ở gần nhau.14	Thất bại khi phải liên kết qua các đoạn cú pháp lồng nhau dài.16
Mật độ ngữ nghĩa	Cao trong mỗi cửa sổ ngữ cảnh.19	Bị hạ thấp một cách giả tạo do định dạng cấu trúc.14

Đánh giá kỹ thuật chia văn bản ngây thơ (naive) trên dữ liệu có cấu trúc

Kiến trúc tiền xử lý mặc định của phần lớn hệ thống truy hồi dựa vào chia văn bản ngây thơ (naive text splitting).⁸ Các thuật toán này—được triển khai rộng rãi trong nhiều framework qua các công cụ như CharacterTextSplitter hay RecursiveCharacterTextSplitter—phân đoạn tài liệu theo ngưỡng ký tự hoặc token cố định.²⁰ Để giảm rủi ro cắt đứt từ hoặc ý tứ tức thời, chúng thường dùng cơ chế “cửa sổ trượt” với tham số chồng lấn (overlap), ví dụ chia thành các chunk 512 token với chồng lấn 50 token.²⁰

Ưu điểm chính của chia văn bản ngây thơ là: thực thi mang tính xác định (deterministic), tốc độ xử lý cao, và áp dụng được cho nhiều loại dữ liệu khác nhau.²⁰ Nó không cần biết trước schema của tài liệu, nên hấp dẫn trong giai đoạn thử nghiệm/prototype.²⁰ Ngoài ra, các đánh giá thực nghiệm do nhà cung cấp cơ sở dữ liệu vector thực hiện cho thấy tách ký tự kiểu đệ quy với 400–512 token và overlap 10%–20% có thể đạt recall 85%–90% trên các tập văn bản không cấu trúc, mục đích chung.⁴

Tuy nhiên, khi áp dụng cho các tài liệu giàu thực thể và có cấu trúc cao như resume JSON, các bộ chia ngây thơ gây ra nhiều chế độ lỗi nghiêm trọng làm tê liệt độ chính xác truy hồi.⁵ Hệ quả nặng nhất là phá hủy ranh giới ngữ nghĩa một cách tùy tiện. Vì hoạt động dựa trên ngưỡng token cứng, chúng không “biết” điểm bắt đầu/kết thúc hợp lý của cấu trúc dữ liệu.²¹ Một chunk kích thước cố định có thể cắt ngang giữa một mảng JSON mô tả lịch sử công việc.⁸ Khi đó, chức danh và tên công ty có thể nằm ở Chunk A, còn các gạch đầu dòng quan trọng về thành tựu và stack kỹ thuật lại bị cô lập ở Chunk B.⁵ Nếu người dùng doanh nghiệp truy vấn ứng viên có thành tựu kỹ thuật cụ thể, máy truy hồi có thể lấy Chunk B do độ tương tự cosine cao với truy vấn, nhưng mô hình sinh sẽ không thể xác định ứng viên nào đã làm việc đó hoặc làm tại công ty nào.²²

Hơn nữa, cơ chế overlap của cửa sổ trượt hoàn toàn không đủ để giải quyết các phụ thuộc ngữ cảnh tầm xa.¹⁰ Overlap 100 token có thể giữ được một câu vắt qua ranh giới, nhưng không thể nối được khoảng cách giữa metadata toàn cục của ứng viên (thường nằm đầu file JSON) và một kỹ năng nằm cuối tài liệu dài.²² Chunking ngây thơ mặc định giả định ý nghĩa ngữ nghĩa có tính cục bộ; trong resume có cấu trúc, ý nghĩa lại mang bản chất

phân cấp và phân tán.²³ Cuối cùng, dùng naive text splitter cho JSON gần như chắc chắn tạo ra một phần lớn vector chứa các mảnh dữ liệu mất ngữ cảnh (orphan chunks).¹⁰

Chunking theo nút cấu trúc: Căn chỉnh phân đoạn theo schema

Để chống lại việc phá vỡ ngữ nghĩa do giới hạn token tùy tiện, các kiến trúc truy hồi nâng cao chuyển từ thuật toán ngây thơ sang chunking theo nút cấu trúc (node-based structural chunking).²⁴ Phương pháp này phân đoạn dựa trên bố cục, schema hoặc cú pháp vốn có của tệp nguồn, thay vì dựa trên số ký tự trừu tượng.²¹ Với resume JSON có cấu trúc cao, cách làm điển hình là dùng các parser chuyên biệt như RecursiveJsonSplitter hoặc HierarchicalNodeParser để duyệt tài liệu như một cây các đối tượng/mảng lồng nhau.²⁶

Logic vận hành của recursive JSON splitter là duyệt theo chiều sâu (depth-first).²⁶ Thuật toán cố gắng giữ nguyên các đối tượng JSON “trọn vẹn”—ví dụ một dictionary đại diện cho một vị trí công việc cụ thể—trong cùng một chunk.²⁶ Chỉ khi một đối tượng lồng nhau vượt quá giới hạn token tối đa của mô hình embedding, nó mới tiếp tục chia nhỏ theo cách đệ quy, đồng thời quản lý ranh giới để khóa con vẫn gắn với khóa cha trực tiếp.²⁶ Nhờ căn chỉnh ranh giới chunk theo ranh giới logic của schema JSON, hệ thống đảm bảo chức danh, thời gian làm việc và trách nhiệm liên quan không bị tách rời một cách tùy tiện.²⁷

Hierarchical node parsing nâng cấp ý tưởng này bằng cách mô hình hóa tương minh quan hệ giữa các tầng cấu trúc.²⁶ Khi parse một resume, thuật toán tạo một nút cha cấp cao chứa dữ liệu tóm tắt, và nhiều nút con chứa chi tiết như dự án hay chứng chỉ.²⁸ Trong cơ sở dữ liệu vector, mỗi nút con giữ tham chiếu theo chương trình tới nút cha.²⁶ Điều này cho phép chiến lược “auto-merging retrieval”: hệ thống tìm kiếm trên các chunk con chi tiết để khớp ngữ nghĩa chính xác, nhưng khi truy hồi thì đưa chunk cha rộng hơn vào prompt để tăng tối đa ngữ cảnh cho mô hình sinh.²⁸

Dù chunking theo nút cấu trúc loại bỏ việc cắt giữa đối tượng dữ liệu, nó vẫn chưa tự động giải quyết thiếu hụt ngữ cảnh ở cấp vĩ mô.⁵ Một đối tượng JSON mô tả dự án kỹ thuật phần mềm có thể “đúng cấu trúc”, nhưng nếu bị truy hồi riêng lẻ, mô hình sinh vẫn thiếu các định danh toàn cục—như tên ứng viên và mức độ seniority—để tổng hợp câu trả lời đầy đủ và chính xác.⁵ Do đó, parsing cấu trúc là điều kiện cần, nhưng chưa đủ nếu không bổ sung ngữ cảnh toàn cục.

Bảng 2. So sánh chunking ngây thơ và chunking theo nút phân cấp

Chỉ số	Chunking theo token ngây thơ	Chunking theo nút phân cấp
Logic phân đoạn	Số token cứng, cố định. ²⁰	Parse schema JSON theo chiều sâu. ²⁶

Giữ ngữ cảnh	Dựa vào overlap tuần tự.21	Dựa vào ánh xạ tham chiếu cha-con.26
Toàn vẹn ranh giới	Thường xuyên tách rời các điểm dữ liệu liên quan.5	Giữ chặt nhóm đối tượng logic.26
Chiến lược truy hồi	Bơm trực tiếp chunk vào prompt.2	Khớp nút con, nhưng bơm nút cha vào prompt.28
Chế độ lỗi chính	Sinh ra “mảnh mò côi”.22	Thiếu metadata toàn cục khi chỉ có nút con.5

Tăng cường ngữ cảnh: Cơ chế bơm metadata toàn cục

Lời giải cho bài toán thiếu ngữ cảnh toàn cục trong truy hồi tài liệu có cấu trúc là tăng cường ngữ cảnh theo chương trình (programmatic context enrichment).30 Chiến lược này trích xuất động các định danh quan trọng, cấp cao từ nút gốc của tài liệu và bơm chúng có hệ thống vào nội dung của mọi chunk con trước khi embedding.5 Nhờ vậy, các mảnh dữ liệu rời rạc được chuyển thành các đơn vị truy hồi tự chứa, đủ ngữ cảnh.30

Trong xử lý resume JSON, kiến trúc thường theo chuỗi thao tác sau: trong giai đoạn parse ingestion ban đầu, một script xác định quét phần gốc JSON để trích xuất một schema metadata toàn cục được định nghĩa sẵn.5 Thông thường bao gồm: họ tên ứng viên, chức danh hiện tại, tổng số năm kinh nghiệm, vị trí địa lý, và một mảng ngắn các năng lực cốt lõi.32 Quá trình này không cần suy luận ML; đơn thuần là ánh xạ theo chương trình các khóa JSON đã biết.33

Sau khi phân đoạn cấu trúc thành các nút con (ví dụ từng công việc, từng bằng cấp), metadata toàn cục được định dạng thành một tiền tố (prefix) chuẩn hóa, tiết kiệm token.30 Tiền tố này sau đó được nối cố định vào chuỗi văn bản của từng nút con.5

Ví dụ một chunk cấu trúc thô cho một vai trò riêng lẻ:

"Role: Senior Systems Architect. Company: CloudCorp. Responsibilities: Engineered distributed low-latency transaction databases."

Sau khi bơm metadata theo chương trình, chunk được biến đổi thành payload giàu ngữ cảnh:

"Candidate: Jonathan Evans | Total Experience: 12 Years | Core Skills: C++, Rust, Distributed Systems | Section: Professional Experience | Role: Senior Systems Architect. Company: CloudCorp. Responsibilities: Engineered distributed low-latency transaction databases."

Tác động toán học lên biểu diễn vector

Phương pháp tăng cường này thường dành 15%–25% giới hạn token của mỗi chunk cho tiền tố metadata toàn cục.³⁰ Phân bổ này làm thay đổi “địa hình toán học” của embedding vector.¹⁸ Vì các mô hình embedding hiện đại tính toán biểu diễn thông qua self-attention hai chiều, việc thêm tên ứng viên và thuộc tính hồ sơ cốt lõi sẽ ảnh hưởng đến điểm attention của mọi token phía sau trong chunk.¹⁵ Kết quả là vector không còn chỉ biểu diễn “kỹ thuật cơ sở dữ liệu”, mà được gom cụm ngữ nghĩa quanh “Jonathan Evans” và “kỹ sư hệ thống phân tán cấp cao”.⁵

Nhiều đánh giá thực nghiệm xác nhận hiệu quả của cách làm này. Các nghiên cứu về chunking có bom metadata cho thấy chunking đệ quy kết hợp bom metadata theo chương trình và embedding có trọng số TF-IDF thường vượt baseline chỉ dựa trên nội dung: đạt precision 82,5% so với 73,3% của semantic content chuẩn.¹⁸ Ngoài ra, các chiến lược “prefix-fusion” có thể đẩy Hit Rate@10 lên tới 0,925 trong các kịch bản truy hồi phức tạp.¹⁸ Khi mỗi mảnh resume đều mang theo ngữ cảnh toàn cục, pipeline truy hồi gần như đảm bảo mô hình sinh có đủ thực thể cần thiết để tạo câu trả lời có cơ sở, hạn chế ảo giác.⁵

Khuôn mẫu nâng cao I: Contextual Retrieval của Anthropic

Mặc dù bom metadata theo chương trình rất hiệu quả cho JSON có cấu trúc chặt, ngành công nghiệp vẫn theo đuổi các lời giải phổ quát để khắc phục mất ngữ cảnh trên cả dữ liệu có cấu trúc và không cấu trúc.³⁵ Phương pháp sinh ngữ cảnh nổi bật nhất là Contextual Retrieval, được Anthropic giới thiệu vào cuối năm 2024.³⁶

Contextual Retrieval dựa trên nguyên lý: cách chính xác nhất để “đặt” một chunk vào đúng vị trí trong tài liệu là dùng LLM để viết rõ một bản tóm tắt ngữ cảnh cho từng phân đoạn.³⁷ Trong pha ingestion/tiền xử lý, hệ thống tách một chunk và đưa nó vào một LLM gọn (ví dụ Claude 3 Haiku), kèm theo toàn bộ tài liệu nguồn.³⁸ Mô hình được điều khiển bằng template prompt để tạo ra một chuỗi ngữ cảnh ngắn (50–100 token) giải thích chunk đó thuộc phần nào và có vai trò gì trong toàn văn.³⁵

Chẳng hạn, với một chunk về bằng sáng chế, LLM có thể tạo: “Đoạn này thuộc mục sở hữu trí tuệ trong CV của TS. Sarah Jenkins, mô tả một bằng sáng chế học máy mà cô là tác giả khi làm Trưởng nhóm Nghiên cứu tại AI Labs năm 2022.”³⁷ Chuỗi ngữ cảnh này được prepend vào chunk thô, sau đó toàn bộ được embedding và lưu vào vector database.³⁶

Để đạt độ trung thực truy hồi tối đa, khung Contextual Retrieval yêu cầu kiến trúc tìm kiếm lai (hybrid search).³⁹ Các chunk đã tăng cường được embedding vào không gian vector dày (dense) để tìm tương tự ngữ nghĩa, đồng thời được lập chỉ mục bằng Contextual BM25 (Best Matching 25) cho tìm kiếm thưa (sparse) theo từ khóa.³⁵ BM25 đánh giá TF-IDF, rất quan trọng với resume vì đảm bảo truy hồi chính xác các từ khóa

hiếm và đặc thù như tên nền tảng độc quyền, acronym chứng chỉ, hoặc tên trường đại học mà dense vector có thể bỏ sót.³⁵ Kết quả từ retriever dense và sparse được gộp, loại trùng, và đưa qua mô hình reranking kiểu cross-encoder để chọn các chunk liên quan nhất.³⁹

Chỉ số hiệu năng và tính khả thi kinh tế

Kết quả thực nghiệm của phương pháp Anthropic cho thấy bước nhảy lớn về độ chính xác truy hồi. Trong các bài test chuẩn hóa, chỉ riêng Contextual Embeddings đã giảm 35% tỉ lệ thất bại truy hồi ở top-20 chunk.³⁹ Khi kết hợp thêm Contextual BM25 trong hybrid search, tỉ lệ thất bại giảm 49%.³⁹ Bổ sung một lớp reranking cuối cùng giúp giảm thất bại tới 67%, kéo tỉ lệ thất bại tuyệt đối xuống còn 1,9%.⁴⁰

Rào cản chính để doanh nghiệp áp dụng Contextual Retrieval là chi phí tính toán cực lớn khi phải chạy suy luận LLM cho từng chunk trên hàng triệu resume.³⁵ Để khả thi về chi phí, Anthropic tích hợp cơ chế prompt caching.³⁸ Vì tài liệu nguồn giống nhau cho mọi chunk trong cùng một tài liệu, toàn văn được nạp vào cache của LLM một lần.³⁸ Các lần tạo ngữ cảnh sau đọc từ cache với mức chiết khấu token rất lớn—thường đạt giảm chi phí ~90%.³⁵ Với một tài liệu 8.000 token chia thành các chunk 800 token, mỗi chunk sinh 100 token ngữ cảnh, chi phí trung bình của ingestion giảm còn khoảng 1,02 USD trên mỗi 1 triệu token tài liệu.³⁹ Với tổ chức ưu tiên độ chính xác tuyệt đối hơn độ trễ ingestion, Contextual Retrieval là đỉnh cao của tiền xử lý sinh ngữ cảnh.⁴¹

Khuôn mẫu nâng cao II: Late Chunking của Jina AI

Nếu Anthropic giải quyết mất ngữ cảnh bằng văn bản ngữ cảnh sinh ra, Jina AI lại tiếp cận bằng cách tái cấu trúc kiến trúc embedding, giới thiệu kỹ thuật Late Chunking.¹⁷ Ý tưởng nhận ra lỗi cốt lõi của pipeline truy hồi truyền thống: việc cắt văn bản trước khi đưa vào mô hình embedding đã phá hỏng sớm các tính toán attention hai chiều mà transformer cần để hiểu ngữ cảnh toàn tài liệu.⁴³

Late Chunking đảo ngược trình tự: đưa toàn bộ tài liệu chưa phân đoạn vào mô hình embedding trước, rồi mới áp dụng thuật toán chunking sau suy luận (post-inference) trên các embedding theo token.⁴⁵

Cơ chế embedding có điều kiện (conditional embeddings)

Kiến trúc này dựa vào các mô hình embedding có ngữ cảnh dài (long-context), ví dụ jina-embeddings-v2-base-en, có thể nhận tối đa 8.192 token.⁴⁵ Khi đưa một tài liệu dài (như CV học thuật chi tiết) vào transformer, các lớp self-attention tính toán quan hệ hình học giữa mọi token đồng thời.¹⁵

Nhờ attention toàn cục, một token ở cuối tài liệu (ví dụ đại từ “her” trong “Led her department’s cloud migration”) được “ràng buộc toán học” với thực thể nêu ở đầu tài liệu (ví dụ “Dr. Emily Chen”).¹⁵ Transformer xử lý toàn bộ chuỗi token và sinh ra dãy

embedding theo token.⁴¹ Quan trọng là mỗi embedding token là “có điều kiện”: nó không chỉ mang nghĩa cục bộ, mà còn phản ánh vị trí ngữ nghĩa của token đó so với toàn tài liệu.¹⁷

Sau khi sinh ra dãy embedding token có ngữ cảnh, hệ thống áp các tín hiệu ranh giới (boundary cues) đã định nghĩa trước.⁴⁷ Các ranh giới này có thể lấy từ splitter JSON để quy hoặc tokenizer theo câu.¹⁷ Vector cuối cho mỗi chunk được tính bằng mean pooling trên đúng đoạn span embedding token thuộc ranh giới đó.¹⁷

Đánh đổi kiến trúc (trade-offs)

Vector chunk thu được đồng thời giữ chi tiết cục bộ và ngữ cảnh toàn cục của resume, loại bỏ nhu cầu cửa sổ trượt tùy tiện hoặc tóm tắt ngữ cảnh bằng LLM vốn tốn kém.⁴⁴ Thử nghiệm cho thấy Late Chunking vượt trội naive chunking trên các thước đo truy hồi exact-match và semantic, đặc biệt với dữ liệu có phụ thuộc ngữ cảnh tầm xa.¹⁷

Tuy nhiên, Late Chunking tạo áp lực tính toán rất lớn ở pha ingestion. Vì độ phức tạp thời gian và bộ nhớ của self-attention tăng theo bình phương độ dài chuỗi (xấp xỉ $O(n^2)$), việc tính embedding cho một tài liệu 8.000 token đòi hỏi GPU memory nhiều hơn đáng kể so với xử lý theo batch các chunk 500 token.⁴⁶ Độ trễ truy hồi lúc query hầu như không đổi (vì vector database vẫn lưu vector chunk chuẩn), nhưng chi phí ingestion ban đầu có thể trở nên không khả thi với tổ chức phải xử lý luồng dữ liệu lớn, tốc độ cao.⁴¹

Bảng 3. So sánh Contextual Retrieval và Late Chunking

Đặc tính	Anthropic Contextual Retrieval	Jina AI Late Chunking
Cơ chế ngữ cảnh	Tăng cường bằng văn bản sinh từ LLM. ³⁸	Self-attention toàn tài liệu (ngữ cảnh hóa bằng toán học). ¹⁷
Thứ tự pipeline	Chunk → LLM tạo ngữ cảnh → Embed → Lưu. ³⁵	Embed toàn tài liệu → Pool theo span → Lưu. ⁴⁵
Chi phí ingestion chính	Chi phí token API cho suy luận LLM. ³⁹	GPU memory cao (tăng theo $O(n^2)$). ⁴⁶
Xử lý nhiều JSON	LLM có thể diễn giải và tóm tắt JSON. ³⁵	Mô hình phải attention qua cú pháp thô. ¹⁷
Chiến lược tối ưu	Prompt caching để giảm chi phí. ³⁸	Dùng encoder long-context chuyên dụng. ⁴⁴

Tổng hợp thực hành tốt nhất trong ngành: Kiến trúc “0-LLM-cost” ở pha chunking

Cả Contextual Retrieval và Late Chunking đều đầy giới hạn năng lực RAG, nhưng triển khai ở quy mô doanh nghiệp (hàng triệu resume) thường xung đột với ràng buộc ngân sách. Hơn nữa, các kỹ thuật này chủ yếu thiết kế để suy ra ngữ cảnh từ văn bản không cấu trúc, trong khi resume dạng JSON vốn đã có cấu trúc tường minh theo chương trình.⁸

Tổ chức muốn đạt “điểm biên hiệu quả” của RAG—tối đa hóa độ chính xác truy hồi, đồng thời đưa chi phí suy luận LLM trong chunking về 0—nên triển khai pipeline ingestion mang tính xác định và nhận biết cấu trúc.⁴⁹ Chuỗi khuyến nghị sau tổng hợp các thực hành tốt nhất để vector hóa dữ liệu CV có cấu trúc.

1. Chuyển đổi sang Markdown theo chương trình

Mẫu phản-thực hành (anti-pattern) nghiêm trọng nhất là vector hóa trực tiếp JSON thô.¹⁴ Để loại bỏ “nhiều cú pháp” làm hỏng embedding, bước ingestion đầu tiên nên parse các mảng JSON và làm phẳng dữ liệu thành Markdown có cấu trúc.¹¹ Markdown áp đặt cấu trúc phân cấp tự nhiên (qua các thẻ #, ##, ###) mà các mô hình embedding hiện đại thường được huấn luyện để nhận diện và ưu tiên.⁵¹ Khi chuyển một đối tượng lồng sâu (ví dụ lịch sử việc làm) thành danh sách Markdown sạch dưới tiêu đề “## Kinh nghiệm làm việc”, hệ thống loại bỏ ngoặc, dấu nháy..., tăng tỉ lệ tín hiệu/nhiều và làm tăng mật độ ngữ nghĩa.¹⁴ Bước này chỉ cần script parse cơ bản và không tốn chi phí suy luận LLM.

2. Phân đoạn ngữ nghĩa nghiêm ngặt theo nút

Sau khi định dạng resume thành Markdown phân cấp, cần loại bỏ hoàn toàn splitter kích thước cố định.⁴ Hệ thống nên dùng các thuật toán chunking “nhận biết tài liệu” như MarkdownNodeParser hoặc splitter tự viết theo regex để phân đoạn đúng theo ranh giới cấu trúc.⁵³ Nhờ vậy, một đơn vị ngữ nghĩa—một mô tả công việc đầy đủ, một bằng cấp, hay một tóm tắt dự án—không bao giờ bị cắt đôi.²¹ Nếu một nút quá dài vượt giới hạn context của mô hình embedding, hãy tách đệ quy ở tầng logic tiếp theo (đoạn văn hoặc gạch đầu dòng) để mỗi vector phản ánh một ý trọn vẹn.²¹

3. Bơm ngữ cảnh xác định (deterministic) với chi phí 0

Để khắc phục “mảnh mœ côi” do tách theo cấu trúc, pipeline cần tăng cường ngữ cảnh với chi phí 0.³⁰ Trong pha parse JSON ban đầu, script trích xuất các định danh metadata toàn cục của ứng viên.³¹ Trước khi gửi chunk vào mô hình embedding, nối một prefix metadata chuẩn hóa vào nội dung chunk.³⁰ Dành khoảng 20% cửa sổ token cho metadata buộc mô hình embedding “gom cụm” vai trò/kỹ năng cụ thể trong hồ sơ toàn cục.³⁰ Cách này đạt hiệu ứng “neo ngữ cảnh” tương tự Anthropic, nhưng hoàn toàn là thao tác tính toán, loại bỏ chi phí 1,02 USD / 1M token cho LLM.³³

4. Tìm kiếm lai Dense/Sparse với Reciprocal Rank Fusion

Do truy vấn của nhà tuyển dụng thường cần khớp chính xác các acronym kỹ thuật hoặc tên trường, chỉ dense vector search là chưa đủ.⁵⁴ Kiến trúc nên lập chỉ mục song song: (1) vector database dense cho tương tự ngữ nghĩa; (2) chỉ mục sparse (như BM25) cho độ chính xác theo từ khóa.⁵⁵ Khi query, lấy top kết quả từ cả hai chỉ mục rồi chuẩn hóa bằng Reciprocal Rank Fusion (RRF).⁵⁵ Thuật toán này tái tính thứ hạng để đảm bảo các chunk đưa vào mô hình sinh vừa phù hợp ý định (semantic) vừa thỏa ràng buộc từ khóa chính xác.⁵⁴

5. Semantic caching để tối ưu thời gian chạy

Trong khi các bước trên loại bỏ chi phí LLM ở ingestion, doanh nghiệp cũng cần giảm chi phí ở pha sinh câu trả lời.⁵⁶ Vì luồng tuyển dụng có nhiều ý định truy vấn lặp lại (ví dụ “tìm PM senior có Agile” và “cho tôi PM Agile cấp senior”), cache theo chuỗi ký tự gần như vô dụng.⁵⁷ Triển khai semantic cache (trên in-memory store như Amazon ElastiCache hoặc Redis) giải quyết sự lặp này.⁵⁸ Khi nhận query, embedding của query được so với embedding của các query đã trả lời.⁵⁷ Nếu cosine similarity vượt ngưỡng nghiêm ngặt (ví dụ 0,95), hệ thống trả ngay câu trả lời đã cache, bỏ qua truy hồi và sinh, giúp giảm độ trễ và giảm chi phí API LLM tới 86%.^{57 58}

Khung đánh giá hệ thống

Chuyển từ naive splitting sang pipeline giàu cấu trúc và bơm metadata đòi hỏi đánh giá định lượng chặt chẽ.⁷ Đánh giá chủ quan bằng người là không đủ để tinh chỉnh logic ranh giới và overlap trên hàng nghìn resume.⁷ Thực hành tốt nhất là tích hợp các framework đánh giá tự động như RAGAS để đo hiệu năng trên tập dữ liệu có ground-truth.⁶⁰

Các chỉ số tối ưu cần theo dõi nghiêm ngặt:

- Context Precision: đo mức tập trung thông tin liên quan trong các chunk truy hồi, đảm bảo việc bơm metadata không làm loãng tín hiệu ngữ nghĩa cốt lõi.⁶¹
- Context Recall: đánh giá hệ thống có truy hồi đủ các chunk cần thiết để trả lời trọn vẹn truy vấn hay không, qua đó kiểm chứng hiệu quả của hybrid BM25 + vector search.⁶¹
- Answer Correctness: thước đo “LLM-as-a-judge” xác nhận mô hình sinh đã sử dụng đúng các giá trị JSON được bảo toàn về cấu trúc để tổng hợp câu trả lời chính xác, hạn chế ảo giác.⁶⁰

Kết luận và khuyến nghị chiến lược

Sự trưởng thành của RAG cho thấy rõ: tính toàn vẹn cấu trúc của dữ liệu nạp vào quyết định thành công cuối cùng của đầu ra sinh. Với các cấu trúc phân cấp phức tạp như resume JSON, tiếp tục dùng splitter ngây thơ là một sai lầm kiến trúc. Chunking kích thước cố định gần như chắc chắn phá ranh giới ngữ nghĩa, làm vỡ quan hệ thực thể, và “đổ” vào vector database các mảnh mớ côi khiến ảo giác và thất bại truy hồi gia tăng.

Chunking theo nút cấu trúc tạo nền tảng cần thiết cho parse resume chính xác bằng cách tôn trọng phân cấp schema và giữ nguyên đối tượng logic. Tuy nhiên, cô lập các nút cấu trúc lại tạo thiếu hụt ngữ cảnh toàn cục. Dù Contextual Retrieval (Anthropic) và Late Chunking (Jina AI) cung cấp lời giải rất “đẹp” về toán học, chúng kéo theo chi phí tính toán và tài chính đáng kể, có thể gây bất ổn cho pipeline ingestion quy mô doanh nghiệp.

Cách tiếp cận chiến lược tối ưu cho CV có cấu trúc là “kỹ nghệ hóa ngữ cảnh” mang tính xác định (deterministic context engineering). Bằng cách chuyển JSON sang Markdown phân cấp, áp chunking nghiêm ngặt theo ranh giới ngữ nghĩa, và bơm metadata toàn cục vào mọi nút con trước embedding, hệ thống có thể buộc nhận thức ngữ cảnh trong không gian embedding. Khi kết hợp với kiến trúc tìm kiếm lai BM25/Vector và semantic caching lúc chạy, pipeline tiền xử lý “0-LLM-cost” này đạt độ chính xác truy hồi rất cao, giúp mô hình sinh được neo vào dữ liệu mạch lạc, đúng cấu trúc và có ngữ cảnh đầy đủ.

Nguồn trích dẫn

- Advanced RAG Techniques for High-Performance LLM Applications - Neo4j, truy cập ngày 03/04/2026, <https://neo4j.com/blog/genai/advanced-rag-techniques/>
- The Ultimate Guide to Chunking Strategies for RAG Applications with Databricks - Medium, truy cập ngày 03/04/2026, <https://medium.com/@debusinha2009/the-ultimate-guide-to-chunking-strategies-for-rag-applications-with-databricks-e495be6c0788>
- Agentforce and RAG: Best Practices for Better Agents - Salesforce, truy cập ngày 03/04/2026, <https://www.salesforce.com/agentforce/agentforce-and-rag/>
- Best Chunking Strategies for RAG (and LLMs) in 2026 - Firecrawl, truy cập ngày 03/04/2026, <https://www.firecrawl.dev/blog/best-chunking-strategies-rag>
- Beyond Fixed Chunks: How Semantic Chunking and Metadata Enrichment Transform RAG Accuracy - Medium (Feb 2026), truy cập ngày 03/04/2026, <https://medium.com/@shaikmohdhuz/beyond-fixed-chunks-how-semantic-chunking-and-metadata-enrichment-transform-rag-accuracy-07136e8cf562>
- Optimize Vector Databases, Enhance RAG-Driven Generative AI - Intel/Medium, truy cập ngày 03/04/2026, <https://medium.com/intel-tech/optimize-vector-databases-enhance-rag-driven-generative-ai-90c10416cb9c>
- Chunking Strategies for Complex RAG Documents (Financial + Legal) - Reddit, truy cập ngày 03/04/2026, https://www.reddit.com/r/Rag/comments/1nf8e1b/chunking_strategies_for_complex_rag_documents/
- The Chunking Strategy That's Killing Your RAG Performance (95% of Developers Get This Wrong) - Medium, truy cập ngày 03/04/2026, <https://medium.com/@theabhishek.040/the-chunking-strategy-thats-killing-your-rag-performance-95-of-developers-get-this-wrong-0600b91daabe>

- PageIndex: The RAG Framework That Threw Out Vector Databases and Still Hit 98.7% Accuracy - Towards AI, truy cập ngày 03/04/2026, <https://pub.towardsai.net/pageindex-the-rag-framework-that-threw-out-vector-databases-and-still-hit-98-7-accuracy-d194e0549478>
- How I Rebuilt a RAG System that Actually Works - Towards AI, truy cập ngày 03/04/2026, <https://pub.towardsai.net/how-i-rebuilt-a-rag-system-that-actually-works-2c5b78437e71>
- Building a Production RAG System for Resume Search: What Actually Worked (and What Didn't) - DEV Community, truy cập ngày 03/04/2026, <https://dev.to/harrywynn/building-a-production-rag-system-for-resume-search-what-actually-worked-and-what-didnt-3jaa>
- How we Chunk - turning PDF's into hierarchical structure for RAG - Reddit, truy cập ngày 03/04/2026, https://www.reddit.com/r/LocalLLaMA/comments/1dpb9ow/how_we_chunk_turning_pdfs_into_hierarchical/
- Retrieval-augmented generation (RAG) failure modes and how to fix them - Snorkel AI, truy cập ngày 03/04/2026, <https://snorkel.ai/blog/retrieval-augmented-generation-rag-failure-modes-and-how-to-fix-them/>
- Optimizing Vector Search: Why You Should Flatten Structured Data - Towards Data Science, truy cập ngày 03/04/2026, <https://towardsdatascience.com/optimizing-vector-search-why-you-should-flatten-structured-data/>
- Late Chunking: Improving RAG Performance with Context-Aware Embeddings - Pondhouse Data, truy cập ngày 03/04/2026, <https://www.pondhouse-data.com/blog/advanced-rag-late-chunking>
- Best practices to help GPT understand heavily nested json data and analyse such data - OpenAI Community, truy cập ngày 03/04/2026, <https://community.openai.com/t/best-practices-to-help-gpt-understand-heavily-nested-json-data-and-analyse-such-data/922339>
- Late Chunking: Contextual Chunk Embeddings Using Long-Context Embedding Models - arXiv, truy cập ngày 03/04/2026, <https://arxiv.org/pdf/2409.04701>
- [2512.05411] A Systematic Framework for Enterprise Knowledge Retrieval: Leveraging LLM-Generated Metadata to Enhance RAG Systems - arXiv, truy cập ngày 03/04/2026, <https://arxiv.org/abs/2512.05411>
- RAG vs. prompt stuffing: Do we still need vector retrieval? - Weights & Biases, truy cập ngày 03/04/2026, <https://wandb.ai/byyoung3/rag-eval/reports/RAG-vs-prompt-stuffing-Do-we-still-need-vector-retrieval---VmlldzoxMzE5Mjk0NA>
- RAG Chunking Strategies & Text Splitters - POMA AI Docs, truy cập ngày 03/04/2026, <https://www.poma-ai.com/docs/rag-chunking-strategies-text-splitters>
- RAG Chunking Strategy - GPT-trainer Blog, truy cập ngày 03/04/2026, <https://gpt-trainer.com/blog/rag+chunking+strategy>

- MultiDocFusion: Hierarchical and Multimodal Chunking Pipeline for Enhanced RAG on Long Industrial Documents - ACL Anthology, truy cập ngày 03/04/2026, <https://aclanthology.org/2025.emnlp-main.1062.pdf>
- Chunking Strategies for LLM Applications - Pinecone, truy cập ngày 03/04/2026, <https://www.pinecone.io/learn/chunking-strategies/>
- Chunking Techniques with Langchain and LlamaIndex - LanceDB, truy cập ngày 03/04/2026, <https://lancedb.com/blog/chunking-techniques-with-langchain-and-llamaindex/>
- How content chunking works for knowledge bases - Amazon Bedrock Docs, truy cập ngày 03/04/2026, <https://docs.aws.amazon.com/bedrock/latest/userguide/kb-chunking.html>
- Develop a RAG Solution - Chunk Enrichment Phase - Azure Architecture Center, truy cập ngày 03/04/2026, <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-enrichment-phase>
- RAG Is Dead. Long Live Context Engineering for LLM Systems - Callstack, truy cập ngày 03/04/2026, <https://www.callstack.com/blog/rag-is-dead-long-live-context-engineering-for-llm-systems>
- A Chunk by Any Other Name: Structured Text Splitting and Metadata-enhanced RAG - LangChain Blog, truy cập ngày 03/04/2026, <https://blog.langchain.com/a-chunk-by-any-other-name/>
- Node Parsers / Text Splitters - LlamaIndex Documentation, truy cập ngày 03/04/2026, https://developers.llamaindex.ai/typescript/framework/modules/data/ingestion_pipeline/transformations/node-parser/
- Finding the Best Chunking Strategy for Accurate AI Responses - NVIDIA Technical Blog, truy cập ngày 03/04/2026, <https://developer.nvidia.com/blog/finding-the-best-chunking-strategy-for-accurate-ai-responses/>
- Contextual Retrieval in AI Systems - Anthropic, truy cập ngày 03/04/2026, <https://www.anthropic.com/news/contextual-retrieval>